

UNIT 4: Section 1: ADO.Net: Sample Code and Examples

Creating and managing connections in ADO.NET

Creating a connection object

The connection component of a data provider establishes a connection with a database. To connect to a Microsoft SQL Server, you use the SQL connection class. The following are the commonly used properties and Methods of the SqlConnection class.

ConnectionString: provides information, such as database name and, user credentials for database access and so on.

Open(): Opens the connection for accessing the database.

Close(): Closes the connection to the database.

For Example

```
// Creating object of SqlConnection Class.
```

```
SqlConnection cn = new SqlConnection();
```

```
// Creating connection string to sample database.
```

```
cn.ConnectionString = "Data source=.; Initial Catalog=Sample; User Id=sa; Password=faculty";
```

The connection string provides the information that defines the connection to the database.

Data Source: Specifies the provider name or your server name.

Initial Catalog: Specifies the name of the database.

User ID and Password: Provide the username and password of your database server.

Open the Connection

```
// Creating object of SqlConnection Class.
```

```
SqlConnection cn = new SqlConnection();
```

```
// Creating connection string to sample database.
```

```
cn.ConnectionString = "Data source=.; Initial Catalog=Sample; User Id=sa; Password=faculty";  
cn.Open(); // it open the connection to database server..
```

Close the Connection

```
// Creating object of SqlConnection Class.
```

```
SqlConnection cn = new SqlConnection();
```

```
// Creating connection string to sample database.
```

```
cn.ConnectionString = "Data source=.; Initial Catalog=Sample; User Id=sa; Password=faculty";
```

```
cn.Open(); // it open the connection to database server..
```

```
// Creating sqlcommand class object
```

```
SqlCommand cmd = new SqlCommand("Select * from tblEmployees", cn);
```

```
SqlDataReader dr = cmd.ExecuteReader(); // Executing query
```

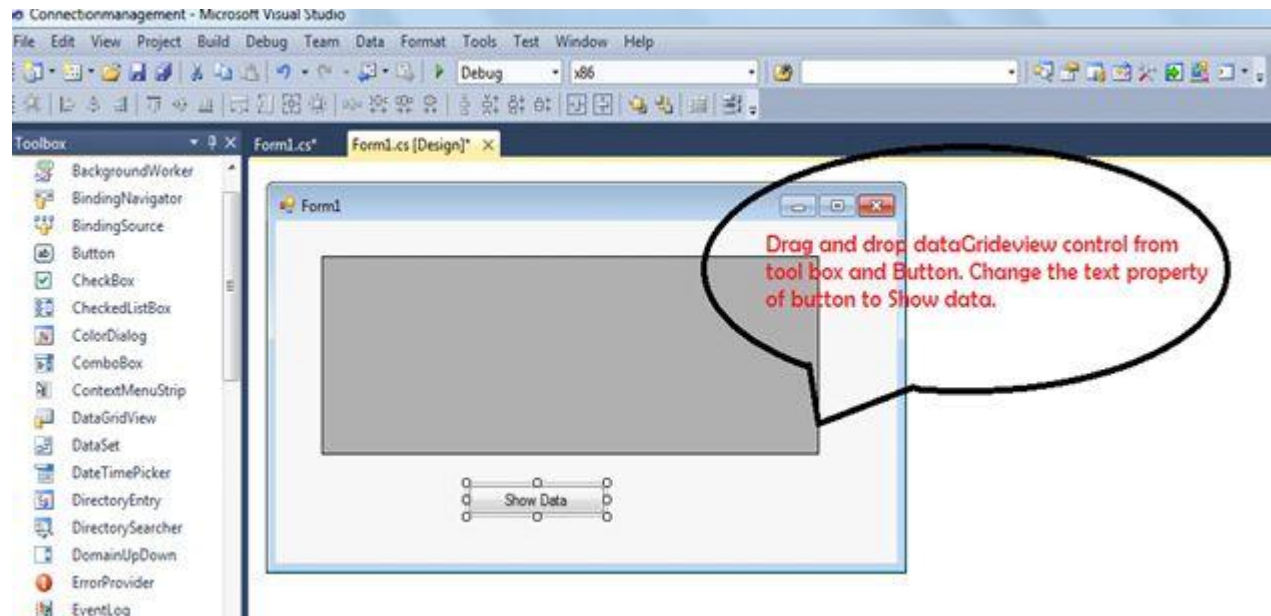
```
cn.Close(); // Closing the connection
```

Demo Application:

Consider the situation in which you are working in XYZ Inc. as an application developer. As a member of the development team, you have been asked to develop an application that will display all the records from the tblEmployee table present in the sample database.

Note: In this example, I'm using a sample database present in my datasource. Here you need to create a sample database and the tblEmployee table in that database and populate the tblEmployee table with some values.

Step 1. Create a Windows Form application and design the form as shown in the following figure.



Step 2. Add the following two namespaces.

System.Data.SqlClient;

System.Data;

Step 3. On click event of the Button write the following code.

```
SqlConnection cn = new SqlConnection();
```

```
// Creating connection string to sample database.
```

```
cn.ConnectionString = "Data source=.; Initial Catalog=Sample; User Id=sa; Password=faculty";
```

```
// Open Connection
```

```
cn.Open();
```

```
// Creating SqlCommand class Object
```

```
SqlCommand cmd = new SqlCommand("Select * from tblEmployee", cn);
```

```
// Creating Object of dataAdapter
```

```
SqlDataAdapter da = new SqlDataAdapter();
```

```
// Creating object of dataSet
```

```
DataSet ds = new DataSet();
```

```
da.SelectCommand = cmd;
```

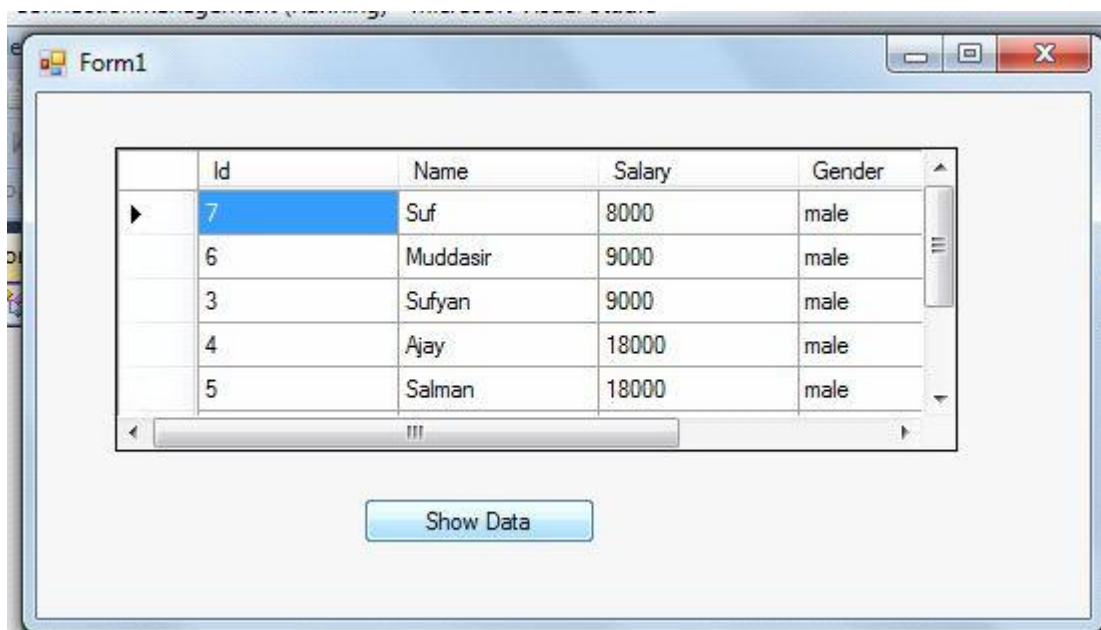
```
// Populating dataset by using fill method of sqlDataAdapter
```

```
da.Fill(ds);
```

```
// Binding dataset with DataGridView1 control
```

```
GridView1.DataSource = ds.Tables[0];
```

Step 4. Execute the application and verify the output by clicking on the button control. If everything goes fine then you will get the following output.



1. Connection String

No.	Connection String Parameter Name	Description
1	Data Source	Identify the server. Could be a local machine, machine domain name, or IP Address.
2	Initial Catalog	Data base name.
3	Integrated Security	Set to SSIP to make a connection with the user's window log in.
4	User ID	Name of user configured in SQL Server.
5	Password	Password matching SQL Server User ID

The connection string is different for each of the various data providers available in .NET 2.0. There are different connection strings for the various types of data sources. You can find a list of all the available providers for creating a connection in a table:

No	Provider	Description
1	System.Data.SqlClient	Provides data for Microsoft SQL Server
2	System.Data.OleDb	Provides data access for data sources exposed using OLE DB
3	System.Data.Odbc	Provides data access for data source exposed using ODBC.
4	System.Data.OracleClient	Provides data access for Oracle.

Example:

```
SqlConnection con;
```

```
con = new SqlConnection("Server=Krushna;Database=Anagha;Uid=sa;Pwd=sa");
```

2. Command Object

A Command object executes SQL statements on the database. These SQL statements can be SELECT, INSERT, UPDATE, or DELETE. It uses a connection object to perform these actions on the database.

A Connection object specifies the type of interaction to perform with the database, like SELECT, INSERT, UPDATE, or DELETE.

A Command object is used to perform various types of operations, like SELECT, INSERT, UPDATE, or DELETE on the database.

SELECT

```
cmd =new SqlCommand("select * from Employee", con);
```

INSERT

```
cmd = new SqlCommand("INSERT INTO Employee(Emp_ID,  
Emp_Name)VALUES ('" + aa + "','" + bb + "')", con);
```

UPDATE

```
SqlCommand cmd =new SqlCommand("UPDATE Employee SET  
Emp_ID='" + aa + "', Emp_Name='" + bb + "' WHERE  
Emp_ID = '" + aa + "'", con);
```

DELETE

```
cmd =new SqlCommand("DELETE FROM Employee where  
Emp_ID='" + aa + "'", con);
```

A Command object exposes several execute methods like:

ExecuteScaler()

Executes the query, and returns the first column of the first row in the result set returned by the query. Extra columns or rows are ignored.

ExecuteReader()

Display all columns and all rows in the client-side environment.

In other words, we can say that they display datatables client-side.

ExecuteNonQuery()

Something is done by the database but nothing is returned by the database.

3. Data Reader Object

A DataReader object is used to obtain the results of a SELECT statement from a command object. For performance reasons, the data returned from a data reader is a forward-only stream of data.

This means that the data can be accessed from the stream in a sequential manner. This is good for speed, but if data needs to be manipulated then a dataset is a better object to work with.

Example

```
dr = cmd.ExecuteReader();
```

```
DataTable dt = new DataTable();
```

```
dt.Load(dr);
```

- It is used in Connected architecture.
- Provide better performance.
- DataReader Object has Read-only access.
- DataReader Object Supports a single table based on a single SQL query of one database.
- While DataReader Object is Bind to a single control.
- DataReader Object has Faster access to data.
- DataReader Object Must be manually coded.
- we can't create a relation in the data reader.
- whereas Data reader doesn't support.
- The data reader communicates with the command object.
- DataReader can not modify data.

4. Data Adapter Object

A Data Adapter represents a set of data commands and a database connection to fill the dataset and update a SQL Server database.

A Data Adapter contains a set of data commands and a database connection to fill the dataset and update a SQL Server database. Data Adapters form the bridge between a data source and a dataset.

Data Adapters are designed depending on the specific data source. The following table shows the Data Adapter classes with their data source.

Provider-Specific Data Adapter classes	Data Source
SqlDataAdapter	SQL Server
OleDbDataAdapter	OLE DB provider
OdbcDataAdapter	ODBC driver
OracleDataAdapter	Oracle

Provider-Specific Data Adapter classes

Data Source

A Data Adapter object accesses data in a disconnected mode. Its object contains a reference to a connection object.

It is designed in a way that implicitly opens and closes the connection whenever required.

It maintains the data in a DataSet object. The user can read the data if required from the dataset and write back the changes in a single batch to the database. Additionally, the Data Adapter contains a command object reference for SELECT, INSERT, UPDATE, and DELETE operations on the data objects and a data source.

A Data Adapter supports mainly the following two methods:

Fill ():

The Fill method populates a dataset or a data table object with data from the database. It retrieves rows from the data source using the SELECT statement specified by an associated select command property.

The Fill method leaves the connection in the same state as it encountered it before populating the data. If subsequent calls to the method for refreshing the data are required then the primary key information should be present.

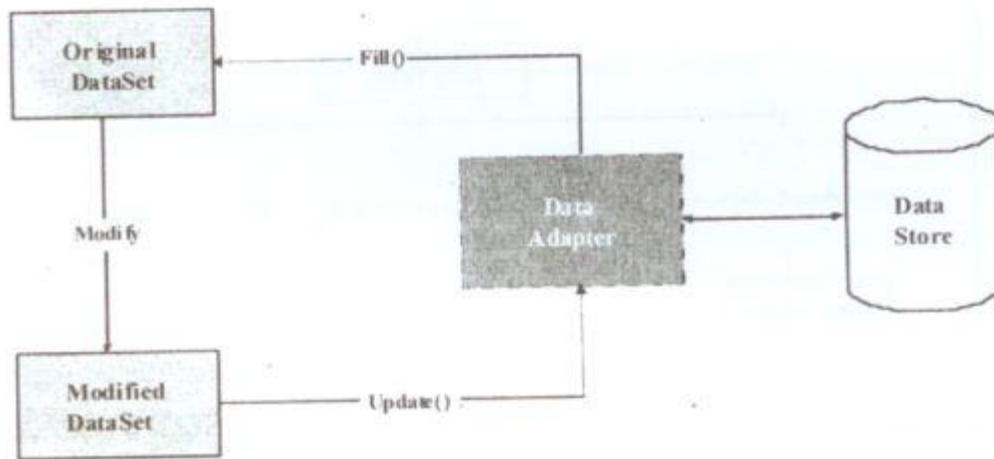
Update ():

The Update method commits the changes back to the database. It also analyzes the RowState of each record in the DataSet and calls the appropriate INSERT, UPDATE, and DELETE statements. A Data Adapter object is formed between a disconnected ADO.NET object and a data source.

Example

```
SqlDataAdapter da=new SqlDataAdapter("Select * from Employee", con);  
da.Fill(ds,"Emp");  
bldr =new SqlCommandBuilder(da);  
dataGridView1.DataSource = ds.Tables["Emp"];
```


Retrieving & updating data using DataAdapter



5. DataSet Object

In the disconnected scenario, the data retrieved from the database is stored in a local buffer called DataSet. It is explicitly designed to access data from any data source. This class is defined in the System.Data namespace.

A Data Set object is an in-memory representation of the data. It is specially designed to manage data in memory and to support disconnected operations on data.

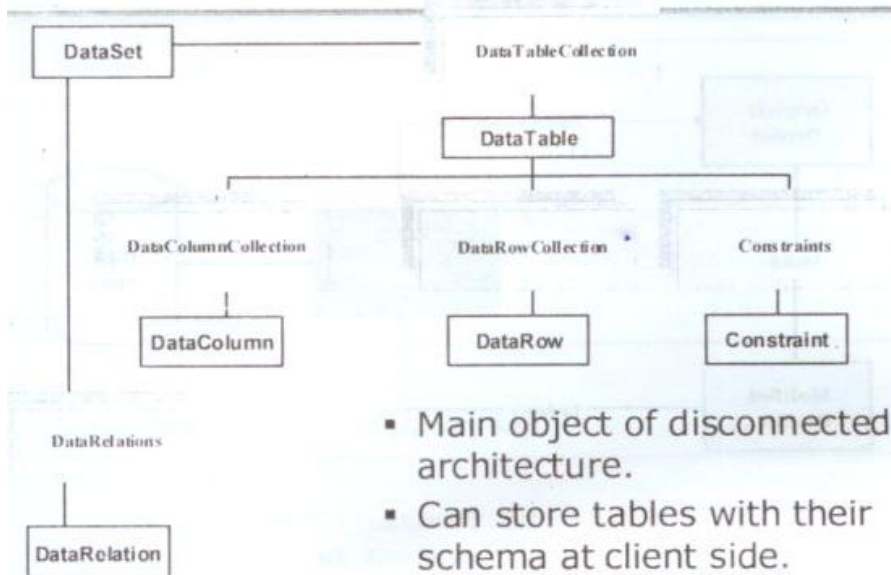
A Data Set is a collection of DataTable and DataRelations. Each DataTable is a collection of DataColumn, DataRows, and Constraints.

A DataTable, DataColumn, and DataRows could be created as follows.

Example

```
DataTable dt = new DataTable();  
DataColumn col = new DataColumn();  
Dt.columns.Add(col2);  
DataRow row = dt.newRow();
```

DataSet object



- It is used in a disconnected architecture.
- Provides lower performance.
- A DataSet object has read/write access.
- A DataSet object supports multiple tables from various databases.
- A DataSet object is bound to multiple controls.
- A DataSet object has slower access to data.
- A DataSet object is supported by Visual Studio tools.
- We can create relations in a dataset.
- A Dataset supports integration with XML.
- A DataSet communicates with the Data Adapter only.
- A DataSet can modify data.

6. Command Builder Object

Automatically generates insert, update, delete queries using the SelectCommand property of a DataAdapter.

A Command Builder Object is used to build commands for data modification from objects based on a single table query. CommandBuilders are designed depending on the specific data source. The following table shows the CommandBuilder classes with their data source.

Provider-Specific Data Adapter classes	Data Source
SqlDataAdapter	SQL Server
OleDbDataAdapter	OLE DB provider
OdbcDataAdapter	ODBC driver
OracleDataAdapter	Oracle

Example

```
da = new SqlDataAdapter("Select * from Employee", con);
ds = new DataSet();
da.MissingSchemaAction = MissingSchemaAction.AddWithKey;
da.Fill(ds, "Emp");
bldr = new SqlCommandBuilder(da);
dataGridView1.DataSource = ds.Tables["Emp"];
```

Differences between DataReader and DataSet

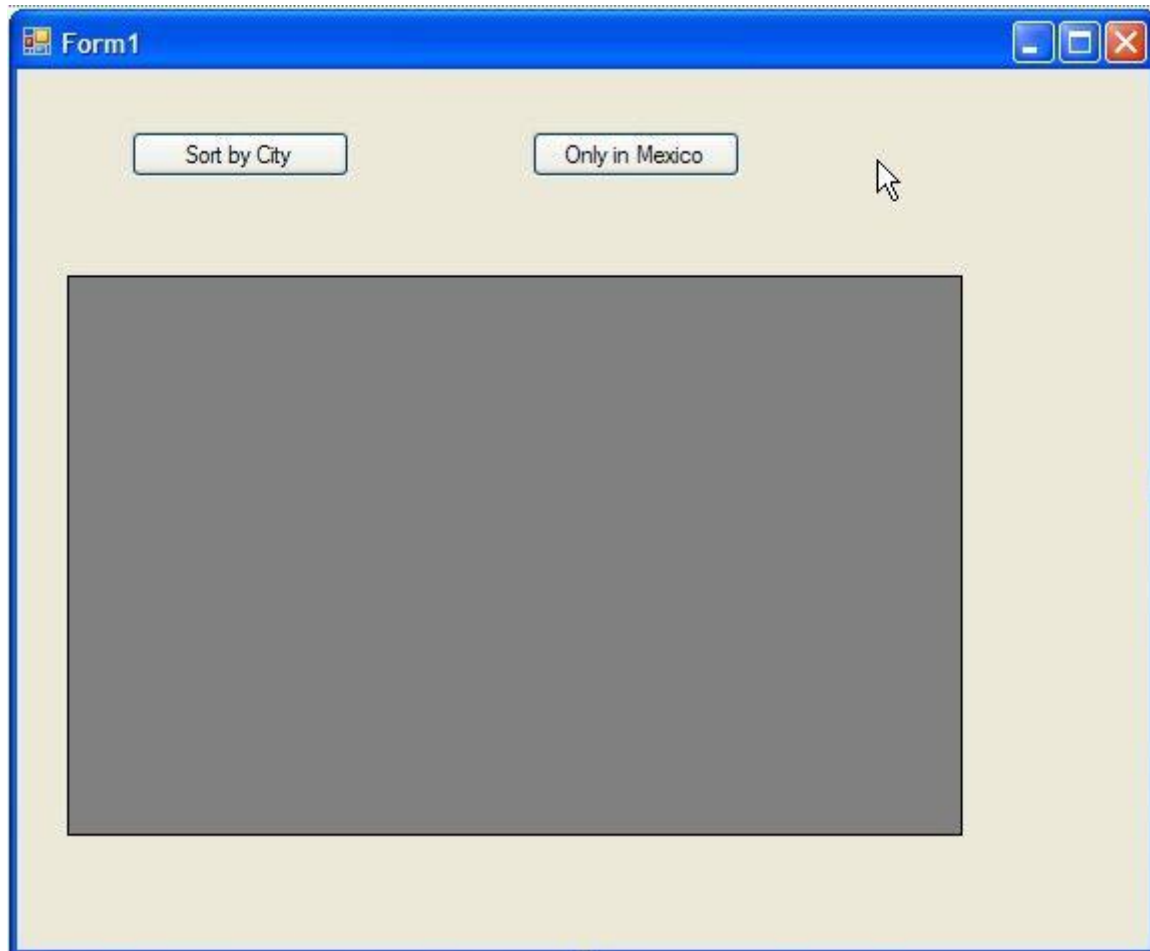
#	Data Reader	DataSet
1	Used in a connected architecture	Used in a disconnected architecture
2	Provides better performance	Provides lower performance
3	DataReader object has read-only access	A DataSet object has read/write access
4	DataReader object supports a single table based on a single SQL query of one database	A DataSet object supports multiple tables from various databases
5	A DataReader object is bound to a single control	A DataSet object is bound to multiple controls
6	A DataReader object has faster access to data	A DataSet object has slower access to data
7	A DataReader object must be manually coded	A DataSet object is supported by Visual Studio tools
8	We can't create a relation in a data reader	We can create relations in a dataset
9	Whereas a DataReader doesn't support data reader communicates with the command object.	A Dataset supports integration with XML Dataset communicates with the Data Adapter only
10	DataReader cannot modify data	A DataSet can modify data

DataView Object

- A DataView is the same as a read-only mini-dataset.
- You typically load only a subset into a DataView.
- A DataView provides a dynamic view of data. It provides a datarow using the DataView.

Example – 1: Add two buttons and a DataGridView control. Change the text of the first button to sort by city and that of button2 to only select records in Mexico and add the code in form.cs.

Form Design



Code:

```
SqlConnection con;  
SqlCommand cmd;  
SqlDataReader dr;  
  
public DataTable GetTable()  
{  
    con = new SqlConnection("Data Source=.\sqlexpress;Initial Catalog=information;Integrated S  
ecurity=True;Pooling=False");
```

```

con.Open();

cmd = new SqlCommand("select * from Customers", con);

dr = cmd.ExecuteReader();

DataTable dt = new DataTable();

dt.Load(dr);

dataGridView1.DataSource = dt;

con.Close();

return dt;
}

```

Form1_Load

```

dataGridView1.DataSource = GetTable().DefaultView;

ShortByCity_Click

DataView dv = new DataView(GetTable());

dv.Sort = "City ASC";

dataGridView1.DataSource = dv;

OnlyInMexico_Click

DataView dv = new DataView(GetTable());

dv.RowFilter = "Country = 'Mexico'";

dataGridView1.DataSource = dv;

```

At the click of the sort by city button, the data already in the DataGridView control is sorted by city.

On clicking the second button, only the records in Mexico are displayed in the DataGridView control. The output after clicking the only in Mexico button is as the Mexico button.

Working With System.Data.SqlClient and System.Data.OleDb

Example-2: Design a simple Winform for accepting the details of Employee. **Using the connected architecture** of ADO.NET, perform the following operations (using System.Data.SqlClient):

- Insert record.
- Search record.
- Update record.
- Delete record.

Form Design:

Emp_ID	Emp_Name	Salary
11	eweew	2323
33	ewew	232
44	werwer	43433
55	kkkkkkk	434343

Code:

Step 1: add namespace using System.Data.SqlClient; for SQL dataBase.

Step 2: Create a connection object.

```
SqlConnection con;
```

```
SqlCommand cmd;
```

```
SqlDataReader dr;
```

Step3:

Form1_Load

```
con =new SqlConnection("Data Source=.\sqlexpress;Initial Catalog=information;Integrated security=True;Pooling=False");
```

private void Display()

```
{  
  
    con.Open();  
  
    cmd =new SqlCommand("select * from Employee", con);  
  
    dr = cmd.ExecuteReader();  
  
    DataTable dt = new DataTable();  
  
    dt.Load(dr);  
  
    dataGridView1.DataSource = dt;  
  
    con.Close();  
  
}
```

Insertbtn_Click

```
con.Open();  
  
int aa = Convert.ToInt32(textBox1.Text);  
  
string bb = textBox2.Text;  
  
int cc = Convert.ToInt32(textBox3.Text);  
  
cmd =new SqlCommand("INSERT INTO Employee(Emp_ID, Emp_Name,Salary)VALUES (" + aa + "," + bb + "," + cc + ")", con);  
  
cmd.ExecuteNonQuery();  
  
MessageBox.Show("one record inserted:");  
  
con.Close();  
  
Display();
```

Deletebtn_Click

```
con.Open();  
int aa = Convert.ToInt32(textBox1.Text);  
cmd =new SqlCommand("DELETE FROM Employee where Emp_ID='" + aa + "'", con);  
cmd.ExecuteNonQuery();  
MessageBox.Show("one record Delete:");  
con.Close();  
Display();
```

Updatebtn_Click

```
con.Open();  
int aa = Convert.ToInt32(textBox1.Text);  
string bb = textBox2.Text;  
int cc = Convert.ToInt32(textBox3.Text);  
string abc = "UPDATE Employee SET Emp_ID ='" + aa + "', Emp_Name ='" + bb + "',Salary  
='\" + cc + '\" WHERE Emp_ID = '\" + aa + '\"";  
SqlCommand cmd =new SqlCommand(abc, con);  
cmd.ExecuteNonQuery();  
MessageBox.Show("one record updated:");  
con.Close();  
Display();
```

Displaybtn_Click

```
Display();
```


Searchbtn_Click

```
con.Open();

int aa = Convert.ToInt32(textBox1.Text);

string abc = "SELECT Emp_ID,Emp_Name,Salary FROM Employee where Emp_ID='" + aa
+ "'";

cmd =new SqlCommand(abc, con);

MessageBox.Show("one record search:");

dr = cmd.ExecuteReader();

DataTable dt =new DataTable();

dt.Load(dr);

dataGridView1.DataSource = dt;

con.Close();
```

Totalrecordbtn_Click

```
con.Open();

cmd = new SqlCommand("select Count(*) from Employee", con);

int a = (int)cmd.ExecuteScalar();

label4.Text = "Total Record:--> " + a.ToString();

con.Close();
```

Exit_Click

```
Application.Exit();
```

Example-3: Design a Simple Winform for accepting the details of an Employee. Using the **connected architecture** of ADO.NET, perform the following operations (using **System.Data.OleDb**):

- Insert record.
- Search record.
- Update record.
- Delete record.

Form Design:

Emp_ID	Emp_Name	Salary
11	eweew	2323
33	ewew	232
44	werwer	43433
55	kkkkkkk	434343

Code:

Step 1: add namespace using System.Data.OleDb; for access Database.

Step 2: Create connection object.

```
OleDbConnection con;
OleDbCommand cmd;
OleDbDataReader dr;
```

Step 3: Form1_Load

```
con = new OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data Source=C:\\Documents and Settings\\Admin\\Desktop\\Ado Connected Demo\\db1.mdb");
```

Private void display()

```
con.Open();
```

```

cmd = new OleDbCommand("select * from Employee",con);
dr = cmd.ExecuteReader();
DataTable dt = new DataTable();
dt.Load(dr);
dataGridView1.DataSource = dt;
con.Close();

```

Display_Click

```

display();

```

Insert_Click

```

con.Open();
int aa = Convert.ToInt32(textBox1.Text);
string bb = textBox2.Text;
int cc = Convert.ToInt32(textBox3.Text);
cmd = new OleDbCommand("INSERT INTO Employee(Emp_ID, Emp_Name,Salary) VALU
ES ('" + aa + "','" + bb + "','" + cc + "')", con);
cmd.ExecuteNonQuery();
MessageBox.Show("one record inserted:");
con.Close();
display();

```

Delete_Click

```

con.Open();
int aa = Convert.ToInt32(textBox1.Text);
cmd = new OleDbCommand("DELETE FROM Employee where Emp_ID='" + aa + "'", con);

cmd.ExecuteNonQuery();
MessageBox.Show("one record Delete:");
con.Close();
display();

```

update_Click

```

con.Open();
int aa = Convert.ToInt32(textBox1.Text);
string bb = textBox2.Text;
int cc = Convert.ToInt32(textBox3.Text);
string abc = "UPDATE Employee SET Emp_ID ='" + aa + "', Emp_Name ='" + bb + "', Salary
=''" + cc + "' WHERE Emp_ID = '" + aa + "'";
OleDbCommand cmd = new OleDbCommand(abc, con);

```

```
cmd.ExecuteNonQuery();  
MessageBox.Show("one record updated:");  
con.Close();  
display();
```

Find_Click

```
con.Open();  
int aa = Convert.ToInt32(textBox1.Text);  
string abc = "SELECT Emp_ID,Emp_Name,Salary FROM Employee where Emp_ID=" + aa  
+ "";  
cmd = new OleDbCommand(abc, con);  
MessageBox.Show("one record search:");  
dr = cmd.ExecuteReader();  
DataTable dt = new DataTable();  
dt.Load(dr);  
dataGridView1.DataSource = dt;  
con.Close();
```

Exit_Click

```
Application.Exit();
```

Example-4: Design a simple Winform for accepting the details of an Employee. Using the **disconnected architecture** of ADO.NET, perform the following operations (using System.Data.SqlClient).

- Insert record.
- Display record.
- Update record.
- Delete record

Form Design

The screenshot shows a Windows Form titled "Form2". On the left side, there are three text boxes labeled "Emp_ID", "Emp_Name", and "Salary". To the right of these is a large, empty rectangular area, likely for displaying data. At the bottom of the form, there are two rows of buttons. The first row contains "Display", "Insert", "Delete", and "Update". The second row contains navigation arrows: "<<", "<", ">", and ">>".

Code:

Step 1: add namespace using System.Data.SqlClient; for SQL dataBase.

Step 2: Create connection object.

```
DataSet ds;  
SqlDataAdapter da;  
SqlConnection con;  
SqlCommandBuilder bldr;
```

Step 3: Form1_Load

```
con = new SqlConnection("Data Source=.\sqlexpress;Initial Catalog=information;IntegratedSecurity=True;Pooling=False");
```

private void display()

```
da = new SqlDataAdapter("Select * from Employee", con);  
ds = new DataSet();  
da.MissingSchemaAction = MissingSchemaAction.AddWithKey;  
da.Fill(ds, "Emp");  
bldr = new SqlCommandBuilder(da);  
dataGridView1.DataSource = ds.Tables["Emp"];
```

Display_Click

```
display();
```

Insert_Click

```
DataRow drnew = ds.Tables["Emp"].NewRow();  
drnew[0] = textBox1.Text;  
drnew[1] = textBox2.Text;  
drnew[2] = textBox3.Text;  
ds.Tables["Emp"].Rows.Add(drnew);  
da.Update(ds, "Emp");  
MessageBox.Show("Record added");  
dataGridView1.DataSource = ds.Tables["Emp"];
```

Delete_Click

```
DataRow row = ds.Tables["Emp"].Rows.Find(Convert.ToInt32(textBox1.Text));  
row.Delete();  
da.Update(ds, "Emp");  
MessageBox.Show("Record Deleted");  
dataGridView1.DataSource = ds.Tables["Emp"];
```

Update_Click

```
DataRow dr = ds.Tables[0].Rows.Find(textBox1.Text);  
dr["Emp_Name"] = textBox2.Text;  
dr["Salary"] = textBox3.Text;  
da.Update(ds, "Emp");  
MessageBox.Show("updated..");  
dataGridView1.DataSource = ds.Tables[0];
```

FirstRcd_Click

```
this.BindingContext[ds.Tables[0]].Position = 0;
```

LastRcd_Click

```
this.BindingContext[ds.Tables[0]].Position = ds.Tables[0].Rows.Count - 1;
```

NextRcd_Click

```
if (this.BindingContext[ds.Tables[0]].Position > 0)
{
    this.BindingContext[ds.Tables[0]].Position -= 1;
}
```

PreviousRcd_Click

```
if (this.BindingContext[ds.Tables[0]].Position < ds.Tables[0].Rows.Count - 1)
{
    this.BindingContext[ds.Tables[0]].Position += 1;
}
```

Sample Code:

1. Connection with SQL Server

Steps:

- Add using System.Data.SqlClient; to your C# file.
- Define a connection string.
- Create a SqlConnection object.
- Open the connection.

Code Example:

```
string connectionString = "Data Source=YourServerName;Initial
Catalog=YourDatabaseName;Integrated Security=True";
SqlConnection connection = new SqlConnection(connectionString);
try
{
    connection.Open();
    MessageBox.Show("Connection successful!");
}
catch (Exception ex)
{
    MessageBox.Show("Error connecting to database: " + ex.Message);
}
finally
{
    if (connection.State == System.Data.ConnectionState.Open)
    {
        connection.Close();
    }
}
```

2. SqlCommand

The SqlCommand object executes SQL statements or stored procedures.

Code Example (Insert):

```
string query = "INSERT INTO YourTable (Column1, Column2) VALUES (@Value1,
@Value2)";
SqlCommand command = new SqlCommand(query, connection);
command.Parameters.AddWithValue("@Value1", "Data1");
command.Parameters.AddWithValue("@Value2", "Data2");
```



```
connection.Open();
int rowsAffected = command.ExecuteNonQuery();
MessageBox.Show(rowsAffected + " row(s) inserted.");
connection.Close();
```

3. DataReader (ExecuteReader)

DataReader retrieves data in a fast, forward-only manner.

Steps:

- Create SqlCommand with a SELECT query.
- Execute command.ExecuteReader().
- Read data using reader.Read() and access columns by name or index.

Code Example:

```
string query = "SELECT Id, Name FROM YourTable";
SqlCommand command = new SqlCommand(query, connection);

connection.Open();
SqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
    int id = reader.GetInt32(0); // or reader["Id"]
    string name = reader.GetString(1); // or reader["Name"]
    MessageBox.Show($"Id: {id}, Name: {name}");
}
reader.Close();
connection.Close();
```

4. ExecuteNonQuery

Used for INSERT, UPDATE, DELETE statements that do not return data. It returns the number of rows affected.

Code Example (Update):

```
string query = "UPDATE YourTable SET Column1 = @NewValue WHERE Id = @Id";
SqlCommand command = new SqlCommand(query, connection);
command.Parameters.AddWithValue("@NewValue", "UpdatedData");
command.Parameters.AddWithValue("@Id", 1);
```

```
connection.Open();
int rowsAffected = command.ExecuteNonQuery();
MessageBox.Show(rowsAffected + " row(s) updated.");
connection.Close();
```

5. ExecuteScalar

Used to retrieve a single value (e.g., count, sum) from the database.

Code Example:

```
string query = "SELECT COUNT(*) FROM YourTable";
SqlCommand command = new SqlCommand(query, connection);

connection.Open();
int count = (int)command.ExecuteScalar();
MessageBox.Show("Total rows: " + count);
connection.Close();
```

6. DataAdapter, DataSet, DataTable, and DataGridView

These objects facilitate disconnected data access and UI binding.

Steps:

- Create SqlDataAdapter with SELECT command.
- Create DataSet or DataTable.
- Use dataAdapter.Fill() to populate the DataSet/DataTable.
- Bind DataGridView to the DataTable.
- Use dataAdapter.Update() to push changes back to the database.

Code Example:

```
string query = "SELECT * FROM YourTable";
SqlDataAdapter dataAdapter = new SqlDataAdapter(query, connection);
DataSet dataSet = new DataSet();

connection.Open();
dataAdapter.Fill(dataSet, "YourTable"); // Fill DataSet with data from "YourTable"
connection.Close();

// Assuming you have a DataGridView named dataGridView1 on your form
dataGridView1.DataSource = dataSet.Tables["YourTable"];
```

```
// To update changes back to the database (after modifying data in DataGridView)
// SqlCommandBuilder commandBuilder = new SqlCommandBuilder(dataAdapter); //
For automatic command generation
// dataAdapter.Update(dataSet, "YourTable");
```

How to connect database with windows form application step by step

Step 1: Set up the SQL Server database

Before you start coding, you'll need a database to work with.

Create a database:

- Open SQL Server Management Studio (SSMS).
- Connect to your server instance (e.g., (localdb)\mssqllocaldb for local development).
- Right-click on Databases and select New Database.
- Name the database AdoNetDemo and click OK.

Create a table:

- Expand the AdoNetDemo database, right-click on Tables, and select New -> Table.
- Define the columns for a Students table:
- Id: int, IDENTITY(1,1), Primary Key
- Name: nvarchar(50)
- Major: nvarchar(50)
- Press Ctrl + S to save the table. Name it Students.

Step 2: Create a C# Windows Forms application

- Open Visual Studio.
- Create a new Windows Forms App (.NET Framework) project.
- Name it AdoNetDemoApp.
- Add the System.Data.SqlClient namespace to your form's code-behind file to use the ADO.NET components.

Step 3: Connect to SQL Server

The connection string holds the information required to connect to the database.

Form Design

- Add a Button to the form.
- Change the button's Text to "Test Connection".

Code Example (Form1.cs)

```
using System.Windows.Forms;

using System.Data.SqlClient; // Required for SQL Server

public partial class Form1 : Form
{
    // Place your connection string here.

    // Replace "YourServerName" with your actual server name.

    private string connectionString = "Data Source=YourServerName;Initial
Catalog=AdoNetDemo;Integrated Security=True;";

    public Form1()
    {
        InitializeComponent();
    }

    private void btnTestConnection_Click(object sender, EventArgs e)
    {
        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            try
            {
                // Open the connection.

                connection.Open();

                MessageBox.Show("Connection Successful!");
            }
            catch (Exception ex)
            {
                // Handle exception
            }
        }
    }
}
```

```

        MessageBox.Show("Connection Failed: " + ex.Message);
    }
} // The 'using' block automatically closes the connection.
}
}

```

Step 4: Execute a command and retrieve data using SqlDataReader

The SqlDataReader provides a fast, efficient way to read data one row at a time. It requires an active connection.

Form Design

- Add a DataGridView control to the form. Name it dataGridView1.
- Add a Button with the text "Load Data".

Code Example (Form1.cs)

```

private void btnLoadData_Click(object sender, EventArgs e)
{
    using (SqlConnection connection = new SqlConnection(connectionString))
    {
        string query = "SELECT Id, Name, Major FROM Students";
        using (SqlCommand command = new SqlCommand(query, connection))
        {
            try
            {
                connection.Open();
                using (SqlDataReader reader = command.ExecuteReader())
                {
                    // Create a new DataTable to hold the data.
                    DataTable dataTable = new DataTable();
                    dataTable.Load(reader); // Loads the data from the reader into the DataTable.
                }
            }
            catch { }
        }
    }
}

```

```

        dataGridView1.DataSource = dataTable; // Binds the DataTable to the DataGridView.
    }
}
catch (Exception ex)
{
    MessageBox.Show("Error loading data: " + ex.Message);
}
}
}
}

```

Step 5: Manipulate data using ExecuteNonQuery

Use ExecuteNonQuery to execute commands that do not return any data, such as INSERT, UPDATE, and DELETE.

Form Design

- Add text boxes for Name and Major. Name them txtName and txtMajor.
- Add a Button with the text "Add Student".

Code Example (Form1.cs)

```

private void btnAddStudent_Click(object sender, EventArgs e)
{
    using (SqlConnection connection = new SqlConnection(connectionString))
    {
        string query = "INSERT INTO Students (Name, Major) VALUES (@Name, @Major)";
        using (SqlCommand command = new SqlCommand(query, connection))
        {
            // Use parameters to prevent SQL injection attacks.
            command.Parameters.AddWithValue("@Name", txtName.Text);
            command.Parameters.AddWithValue("@Major", txtMajor.Text);

```

```

try
{
    connection.Open();

    int rowsAffected = command.ExecuteNonQuery();

    MessageBox.Show($"{rowsAffected} row(s) inserted successfully.");
}
catch (Exception ex)
{
    MessageBox.Show("Error adding student: " + ex.Message);
}
}
}

```

Step 6: Retrieve a single value using ExecuteScalar

ExecuteScalar is designed to retrieve a single value from a database query, such as an aggregate result like COUNT or SUM.

Form Design

- Add a Button with the text "Get Student Count".
- Add a Label to display the count.

Code Example (Form1.cs)

```

private void btnGetCount_Click(object sender, EventArgs e)
{
    using (SqlConnection connection = new SqlConnection(connectionString))
    {
        string query = "SELECT COUNT(*) FROM Students";
        using (SqlCommand command = new SqlCommand(query, connection))
        {

```



```

try
{
    connection.Open();

    object result = command.ExecuteScalar();

    if (result != null)
    {
        int count = Convert.ToInt32(result);

        MessageBox.Show($"Total number of students: {count}");
    }
}
catch (Exception ex)
{
    MessageBox.Show("Error getting count: " + ex.Message);
}
}
}

```

Step 7: Disconnected architecture with SqlDataAdapter and DataSet

This approach is particularly useful for populating a DataGridView and then updating the changes back to the database.

Form Design

- Add two buttons: "Load with Adapter" and "Update Database".
- Keep the existing DataGridView.

Code Example (Form1.cs)

```
// Store the DataSet and Adapter at a class level.
```

```
private DataSet dataSet;
```

```
private SqlDataAdapter dataAdapter;
```

```

private void btnLoadWithAdapter_Click(object sender, EventArgs e)
{
    dataSet = new DataSet();
    using (SqlConnection connection = new SqlConnection(connectionString))
    {
        string query = "SELECT Id, Name, Major FROM Students";
        dataAdapter = new SqlDataAdapter(query, connection);

        // Use a SqlCommandBuilder to automatically generate Insert, Update, and Delete
        // commands.
        SqlCommandBuilder commandBuilder = new SqlCommandBuilder(dataAdapter);
        try
        {
            // Fills the DataSet from the database.
            dataAdapter.Fill(dataSet, "Students");
            // Binds the DataTable to the grid.
            dataGridView1.DataSource = dataSet.Tables["Students"];
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error loading data: " + ex.Message);
        }
    }
}

private void btnUpdateDatabase_Click(object sender, EventArgs e)

```

```
{  
    if (dataAdapter != null && dataSet != null)  
    {  
        try  
        {  
            // Pushes changes from the DataSet back to the database.  
            int rowsAffected = dataAdapter.Update(dataSet.Tables["Students"]);  
            MessageBox.Show($"{rowsAffected} row(s) updated successfully.");  
        }  
        catch (Exception ex)  
        {  
            MessageBox.Show("Error updating database: " + ex.Message);  
        }  
    }  
}
```